Viridis Research — certification record

Run `runs/2026-11-09_P1` 2026-11-09 project P1, task P1-T003 engine `8c1757ff8e36`
GitHub run: `https://github.com/jdhart81/viridis-engine/actions/runs/42665815009`

A machine-checked raster bridge for Trees Cannot Cut: single-pixel insertion preserves enc - C8 + chi

Justin Hart, Viridis LLC

Labels awarded by the verifier

Claim	Label	Claim text (as submitted)
C1	**PROVEN**	formal [P1.Insertion]: For every finite raster B of pixels in Z^2 and every pixel p not in B, enclosed(insert p B) - C8(insert p B) + euler(insert p B) = enclosed(B) - C8(B) + euler(B), with C8 the number of 8-components of B, enclosed the number of finite 4-components of Z^2 minus B, and euler = V - E + F of the cubical complex of B (Candidate.main). The same file also proves, as the auxiliary theorem bridge_eq, that enclosed(B) = C8(B) - euler(B) for every finite raster B.

Sealed statement P1.Insertion

File `lean/ViridisStatements/P1/Insertion.lean`, sha256 `800bd9b9fec5033192b40a9d02dae8abf5c15195b3e2b6f2a1c438a7ab08c734`, sealed by decision D-009. Axioms used by the proof: Classical.choice, Quot.sound, propext. Candidate sha256 `e31434107d76a387f2bd47d3752511581084d3195b2563373ad279fbc907d670`. Listing shown with symbol substitution; the exact bytes are in the repository.

```
import Mathlib

/-!
# P1 · L2: single-pixel insertion

Adding one barrier pixel `p ∉ B` to a finite raster leaves
`enclosed − C8 + χ` unchanged, under (8,4) connectivity. There is no hypothesis
on the 3×3 neighbourhood of `p`: every one of its 256 configurations is covered,
including insertions that merge several 8-components and insertions that close
or split holes.

The definitions of `C8`, `enclosed` and their conventions are identical to those
in the `P1.Bridge` proposal (complement taken in all of `ℤ²`; enclosed means
finite). `χ` is `euler`, restated verbatim from the sealed rung L1 `P1.BitQuad`.

This is the second rung (L2) of the P1 ladder: the inductive step for L3.
-/

namespace ViridisStatements.P1.Insertion

/-! ## Definitions restated from the sealed module `P1.BitQuad`

The proposal environment does not load the sealed module `ViridisStatements.P1.BitQuad`,
so the definitions this statement needs are restated here, character for character
identical to that module (sealed rung L1). Once the module can be imported, these may
be replaced by `open ViridisStatements.P1.BitQuad` without changing any meaning. -/

/-- A pixel, named by its lower-left lattice corner. -/
abbrev Pixel := ℤ × ℤ

/-- The four lattice corners of pixel `p`. -/
def corners (p : Pixel) : Finset (ℤ × ℤ) :=
  {(p.1, p.2), (p.1 + 1, p.2), (p.1, p.2 + 1), (p.1 + 1, p.2 + 1)}

/-- Vertices of `K B`: every corner of every pixel of `B`. -/
def vertices (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion corners

/-- Horizontal unit edges `[(i, j), (i + 1, j)]` of `K B`, named by their left
end. These are the bottom edge `(i, j)` and the top edge `(i, j + 1)` of each
pixel `(i, j)` of `B`. -/
def hEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion fun p => {(p.1, p.2), (p.1, p.2 + 1)}

/-- Vertical unit edges `[(i, j), (i, j + 1)]` of `K B`, named by their lower
end. These are the left edge `(i, j)` and the right edge `(i + 1, j)` of each
pixel `(i, j)` of `B`. -/
def vEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
```

```
  B.biUnion fun p => {(p.1, p.2), (p.1 + 1, p.2)}

/-- The Euler characteristic `V − E + F` of `K B`. -/
def euler (B : Finset Pixel) : ℤ :=
  ((vertices B).card : ℤ) - (((hEdges B).card : ℤ) + (vEdges B).card) + B.card

/-! ## Connectivity on `ℤ²` -/

The adjacency relations are the usual digital-topology ones. `fromRel` symmetrises
the relation and removes loops, so no side proofs are needed. -/

/-- 4-adjacency: pixels that share an edge. -/
def G4 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| + |p.2 - q.2| = 1

/-- 8-adjacency: distinct pixels that share an edge or a corner. -/
def G8 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| ≤ 1 ∧ |p.2 - q.2| ≤ 1

/-- The barrier `B`, read with 8-adjacency. -/
def barrierGraph (B : Finset Pixel) : SimpleGraph (B : Set Pixel) :=
  G8.induce (B : Set Pixel)

/-- The complement `ℤ² \ B` (taken in all of `ℤ²`), read with 4-adjacency. -/
def complementGraph (B : Finset Pixel) : SimpleGraph ((B : Set Pixel)? : Set Pixel) :=
  G4.induce (B : Set Pixel)?

/-- `C8(B)`: the number of 8-components of the barrier. `B` is finite, so this
`Nat.card` is a genuine count. -/
noncomputable def C8 (B : Finset Pixel) : ℕ :=
  Nat.card (barrierGraph B).ConnectedComponent

/-- A 4-component of `ℤ² \ B` is enclosed when it is finite. -/
def IsEnclosed {B : Finset Pixel} (c : (complementGraph B).ConnectedComponent) : Prop :=
  c.supp.Finite

/-- The number of enclosed 4-components of `ℤ² \ B`. (`Nat.card` would read 0 on an
infinite type; `P1.Bridge` therefore also asserts that there are finitely many
enclosed components.) -/
noncomputable def enclosed (B : Finset Pixel) : ℕ :=
  Nat.card {c : (complementGraph B).ConnectedComponent // IsEnclosed c}

/-- The quantity that L2 says is invariant, and that L3 says is zero. -/
noncomputable def invariant (B : Finset Pixel) : ℤ :=
  (enclosed B : ℤ) - C8 B + euler B

/-- **Proposed target `P1.Insertion`.** Inserting any non-barrier pixel into any
finite raster leaves `enclosed − C8 + χ` unchanged. -/
def Statement : Prop :=
  ∀ (B : Finset Pixel) (p : Pixel), p ∉ B → invariant (insert p B) = invariant B

/-! ## A computable bounded-box model (test oracle only)

`Statement` does not mention anything in this section. The definitions below
recompute `C8` and the enclosed count inside a bounding box with a one-pixel
margin, by flood fill, so that the controls can be checked by `decide`. That the
box model agrees with the definitions above on every finite raster is a lemma
for the proving task. It is not claimed here. The control values are also proved
for the semantic definitions themselves at the end of this file. -/

/-- Decidable 4-adjacency (same relation as `G4`). -/
def adj4 (p q : Pixel) : Prop := |p.1 - q.1| + |p.2 - q.2| = 1

/-- Decidable 8-adjacency (same relation as `G8`). -/
def adj8 (p q : Pixel) : Prop := p ≠ q ∧ |p.1 - q.1| ≤ 1 ∧ |p.2 - q.2| ≤ 1

instance : DecidableRel adj4 := fun _ _ => by unfold adj4; infer_instance
instance : DecidableRel adj8 := fun _ _ => by unfold adj8; infer_instance

/-- Lexicographic order on pixels, used to pick one representative per component. -/
def lexLe (p q : Pixel) : Prop := p.1 < q.1 ∨ (p.1 = q.1 ∧ p.2 ≤ q.2)

instance : DecidableRel lexLe := fun _ _ => by unfold lexLe; infer_instance

/-- One flood-fill step inside `A`. -/
def grow (r : Pixel → Pixel → Prop) [DecidableRel r] (A S : Finset Pixel) : Finset Pixel :=
  S ∪ A.filter fun q => ∃ s ∈ S, r s q

/-- Everything in `A` reachable from `S` through `A`; `|A|` steps suffice. -/
def reach (r : Pixel → Pixel → Prop) [DecidableRel r] (A S : Finset Pixel) : Finset Pixel :=
  (grow r A)^[A.card] S

/-- The number of `r`-components of `A`: count the lexicographically least pixel
of each component. -/
def numComponents (r : Pixel → Pixel → Prop) [DecidableRel r] (A : Finset Pixel) : ℕ :=
```

```
  (A.filter fun q => ∀ s ∈ reach r A {q}, lexLe q s).card

/-- Bounding box of `B ∪ {(0, 0)}`, with a margin of one pixel on every side. -/
def xlo (B : Finset Pixel) : ℤ := B.fold min 0 (·.1) - 1
def xhi (B : Finset Pixel) : ℤ := B.fold max 0 (·.1) + 1
def ylo (B : Finset Pixel) : ℤ := B.fold min 0 (·.2) - 1
def yhi (B : Finset Pixel) : ℤ := B.fold max 0 (·.2) + 1
def box (B : Finset Pixel) : Finset Pixel :=
  Finset.Icc (xlo B) (xhi B) ×? Finset.Icc (ylo B) (yhi B)

/-- Non-barrier pixels of the box that reach the box corner, which lies in the
unbounded component. -/
def exterior (B : Finset Pixel) : Finset Pixel :=
  reach adj4 (box B \ B) {(xlo B, ylo B)}

/-- Non-barrier pixels of the box that do not reach the corner. -/
def holes (B : Finset Pixel) : Finset Pixel := (box B \ B) \ exterior B

/-- Box-model count of enclosed 4-components. -/
def boxEnclosed (B : Finset Pixel) : ℕ := numComponents adj4 (holes B)

/-- Box-model count of 8-components of the barrier. -/
def boxC8 (B : Finset Pixel) : ℕ := numComponents adj8 B

/-! ## Control rasters -/

/-- Witness: the 3×3 ring, eight pixels around the empty centre `(1, 1)`. -/
def ring : Finset Pixel := {(0, 0), (1, 0), (2, 0), (0, 1), (2, 1), (0, 2), (1, 2), (2, 2)}

/-- Negative control 1: the solid 2×2 block. Its 8-adjacency graph is `K₄`. -/
def block : Finset Pixel := {(0, 0), (1, 0), (0, 1), (1, 1)}

/-- Negative control 2: the four pixels N, S, E, W of the empty centre `(1, 1)`.
Their 4-adjacency graph has no edges, but under (8,4) they enclose the centre. -/
def diamond : Finset Pixel := {(1, 0), (0, 1), (2, 1), (1, 2)}

/-- Box-model version of `invariant`, computable. -/
def boxInvariant (B : Finset Pixel) : ℤ :=
  (boxEnclosed B : ℤ) - boxC8 B + euler B

/-! ## Sanity checks by `decide` (box model)

The controls of the ladder, and insertions that exercise the non-local cases. -/

set_option maxRecDepth 100000 in
/-- Witness: the 3×3 ring has `C8 = 1`, `χ = 0` and one enclosed component. -/
example : boxC8 ring = 1 ∧ euler ring = 0 ∧ boxEnclosed ring = 1 := by decide

set_option maxRecDepth 100000 in
/-- Negative control 1: the 2×2 block has `C8 = 1`, `χ = 1` and encloses nothing. -/
example : boxC8 block = 1 ∧ euler block = 1 ∧ boxEnclosed block = 0 := by decide

set_option maxRecDepth 100000 in
/-- Negative control 2: the diamond has `C8 = 1`, `χ = 0` and one enclosed
component under (8,4). -/
example : boxC8 diamond = 1 ∧ euler diamond = 0 ∧ boxEnclosed diamond = 1 := by decide

set_option maxRecDepth 100000 in
/-- Closing a hole: inserting `(1, 2)` into the ring with its top middle removed
keeps `C8 = 1`, drops `χ` from 1 to 0 and creates one enclosed component. -/
example : boxInvariant (ring.erase (1, 2)) = 0 ∧ boxInvariant ring = 0 ∧
    boxEnclosed (ring.erase (1, 2)) = 0 ∧ euler (ring.erase (1, 2)) = 1 := by decide

set_option maxRecDepth 100000 in
/-- Merging: the left and right columns of the ring are two 8-components
(`χ = 2`). Inserting `(1, 0)` merges them (`C8 = 1`, `χ = 1`, nothing enclosed). -/
example : boxC8 ({(0, 0), (0, 1), (0, 2), (2, 0), (2, 1), (2, 2)} : Finset Pixel) = 2 ∧
    euler ({(0, 0), (0, 1), (0, 2), (2, 0), (2, 1), (2, 2)} : Finset Pixel) = 2 ∧
    boxInvariant ({(0, 0), (0, 1), (0, 2), (2, 0), (2, 1), (2, 2)} : Finset Pixel) = 0 ∧
    boxInvariant ({(1, 0), (0, 0), (0, 1), (0, 2), (2, 0), (2, 1), (2, 2)} : Finset Pixel) = 0 := by
  decide

set_option maxRecDepth 100000 in
/-- Corner-only merging: inserting the last pixel of the diamond joins three
corner-touching pixels into a cycle and encloses the centre. -/
example : boxInvariant (diamond.erase (1, 2)) = 0 ∧ boxEnclosed (diamond.erase (1, 2)) = 0 ∧
    euler (diamond.erase (1, 2)) = 1 ∧ boxInvariant diamond = 0 := by decide

set_option maxRecDepth 100000 in
/-- Filling a hole: inserting the centre `(1, 1)` into the ring removes the enclosed
component and raises `χ` from 0 to 1, giving the solid 3×3 block. -/
example : boxInvariant (insert (1, 1) ring) = 0 ∧ boxEnclosed (insert (1, 1) ring) = 0 ∧
    euler (insert (1, 1) ring) = 1 := by decide


/-! ## Every configuration of the 3×3 neighbourhood, by `decide` (bitmask model)
```

```
/- `Statement` does not mention anything in this section. It is a second, independent
test oracle, written with natural-number bit operations so that the kernel can check
all 256 configurations of the eight neighbours of an inserted pixel in seconds.

The model lives in a 7×7 box: pixel `(x, y)` with `0 ≤ x, y < 7` is bit `7 * y + x`.
Barrier pixels are confined to the inner 5×5, so the box border is non-barrier and
4-connected, and its 4-component is the unbounded one. `chi` counts `V − E + F` with
the same naming of vertices and edges as `euler` (lower-left corner, left end,
lower end). As with the box model, agreement with the semantic definitions is not
proved here. -/

namespace Mask

/-- All 49 bits of the box. -/
def full : ℕ := 2 ^ 49 - 1

/-- The box without column 0. -/
def notCol0 : ℕ := full - (List.range 7).foldl (fun a y => a + 2 ^ (7 * y)) 0

/-- The box without column 6. -/
def notCol6 : ℕ := full - (List.range 7).foldl (fun a y => a + 2 ^ (7 * y + 6)) 0

/-- Left and right neighbours of a set of pixels (no wrap-around). -/
def horiz (S : ℕ) : ℕ := ((S &&& notCol6) <<< 1) ||| ((S &&& notCol0) >>> 1)

/-- Lower and upper neighbours of a set of pixels, inside the box. -/
def vert (S : ℕ) : ℕ := ((S <<< 7) ||| (S >>> 7)) &&& full

/-- `S` together with its 4-neighbours. -/
def nbr4 (S : ℕ) : ℕ := S ||| horiz S ||| vert S

/-- `S` together with its 8-neighbours. -/
def nbr8 (S : ℕ) : ℕ := let H := S ||| horiz S; H ||| vert H

/-- Flood fill from `S` inside `A`, with fuel. -/
def fill (nb : ℕ → ℕ) (A S : ℕ) : ℕ → ℕ
  | 0 => S
  | n + 1 => let S' := nb S &&& A; if S' = S then S else fill nb A S' n

/-- The lowest set bit of `A`. -/
def lowBit (A : ℕ) : ℕ := A - (A &&& (A - 1))

/-- The number of components of `A`: peel off the component of the lowest bit. -/
def countComp (nb : ℕ → ℕ) : ℕ → ℕ → ℕ
  | 0, _ => 0
  | n + 1, A => if A = 0 then 0 else 1 + countComp nb n (A ^^^ fill nb A (lowBit A) 49)

/-- The number of set bits among the lowest `n`. -/
def popcount : ℕ → ℕ → ℕ
  | 0, _ => 0
  | n + 1, m => m % 2 + popcount n (m / 2)

/-- 8-components of the barrier. -/
def c8 (B : ℕ) : ℕ := countComp nbr8 49 B

/-- 4-components of the non-barrier pixels that do not reach the box corner. -/
def enc (B : ℕ) : ℕ :=
  let C := full ^^^ B
  countComp nbr4 49 (C ^^^ fill nbr4 C 1 49)

/-- `V − E + F`. Vertices are the lower-left corners of the pixels at offsets
(0,0), (1,0), (0,1), (1,1) from a barrier pixel; horizontal edges those at offsets
(0,0), (0,1); vertical edges those at offsets (0,0), (1,0). -/
def chi (B : ℕ) : ℤ :=
  (popcount 64 (B ||| (B <<< 1) ||| (B <<< 7) ||| (B <<< 8)) : ℤ)
    - ((popcount 64 (B ||| (B <<< 7)) : ℤ) + popcount 64 (B ||| (B <<< 1)))
    + popcount 64 B

/-- The model's `enclosed − C8 + χ`. -/
def inv (B : ℕ) : ℤ := (enc B : ℤ) - c8 B + chi B

/-- The inserted pixel `p`, at the centre `(3, 3)` of the box. -/
def centre : ℕ := 2 ^ 24

/-- The bits of the eight neighbours of `p`, in the order SW, S, SE, W, E, NW, N, NE. -/
def nbIdx : List ℕ := [16, 17, 18, 23, 25, 30, 31, 32]

/-- Neighbourhood configuration `m < 256`, as a set of barrier pixels around `p`. -/
def spread (m : ℕ) : ℕ :=
  (List.range 8).foldl (fun a i => if m.testBit i then a ||| 2 ^ (nbIdx.getD i 0) else a) 0

/-- The 16 pixels at Chebyshev distance 2 from `p`. As a context, it 8-connects every
barrier neighbour of `p` to every other one, outside the 3×3 window. -/
def ring5 : ℕ := (List.range 49).foldl (fun a i =>
  let x := i % 7; let y := i / 7
```

```
  if (x = 1 ∨ x = 5) ∧ 1 ≤ y ∧ y ≤ 5 ∨ (y = 1 ∨ y = 5) ∧ 1 ≤ x ∧ x ≤ 5 then a ||| 2 ^ i
  else a) 0

/-- For every configuration `m < n` of the eight neighbours, placed in context `ctx`:
the invariant is `0` before and after inserting `p`, so it is unchanged. -/
def allConfigs (ctx : ℕ) : ℕ → Bool
  | 0 => true
  | n + 1 =>
    let B := ctx ||| spread n
    (inv (B ||| centre) == inv B && inv B == 0) && allConfigs ctx n

/-- Witness in the bitmask model: the eight neighbours of `p` form the 3×3 ring. -/
example : enc (spread 255) = 1 ∧ c8 (spread 255) = 1 ∧ chi (spread 255) = 0 := by decide

/-- Negative control 1 in the bitmask model: the 2×2 block `{p, W, S, SW}`. -/
example : enc (centre ||| spread 0b1011) = 0 ∧ c8 (centre ||| spread 0b1011) = 1 ∧
    chi (centre ||| spread 0b1011) = 1 := by decide

/-- Negative control 2 in the bitmask model: the diamond `{S, W, E, N}` encloses `p`. -/
example : enc (spread 0b01011010) = 1 ∧ c8 (spread 0b01011010) = 1 ∧
    chi (spread 0b01011010) = 0 ∧ countComp nbr4 49 (spread 0b01011010) = 4 := by decide

set_option maxRecDepth 100000 in
/-- **All 256 configurations, alone.** For every subset `N` of the eight neighbours of
`p`, inserting `p` into `N` leaves `enclosed — C8 + χ` unchanged (and equal to `0`).
This covers all 512 subsets of the 3×3 window. -/
example : allConfigs 0 256 = true := by decide

set_option maxRecDepth 100000 in
/-- **All 256 configurations, inside a closed ring.** The same, with the 5×5 ring `ring5`
added as context. Now all barrier neighbours of `p` are already 8-connected through
the ring, so inserting `p` never merges 8-components, and every non-barrier run around
`p` lies in a finite region. -/
example : allConfigs ring5 256 = true := by decide

end Mask

/-! ## The controls for the semantic definitions

The checks above use the box model. The lemmas below prove the same control
values for `C8` and `enclosed` themselves (the definitions that `Statement`
uses), with no appeal to the box model: a horizontal ray of non-barrier pixels
makes a component infinite, and a non-barrier pixel whose four 4-neighbours are
barrier is a finite component on its own. -/

namespace Controls

/-- A horizontal ray of non-barrier pixels makes a component infinite. -/
theorem infinite_of_ray (B : Finset Pixel) (v : Pixel) (hv : v ∉ B) (d : ℤ) (hd : d = 1 ∨ d = -1)
    (hray : ∀ n : ℕ, (v.1 + d * n, v.2) ∉ B) :
    ¬ IsEnclosed ((complementGraph B).connectedComponentMk ⟨v, hv⟩) := by
  intro hfin
  have hreach : ∀ n : ℕ, (complementGraph B).Reachable ⟨v, hv⟩ ⟨(v.1 + d * n, v.2), hray n⟩ := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      refine ih.trans (SimpleGraph.Adj.reachable ?_)
      simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
        SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype, ne_eq, Prod.mk.injEq]
      push_cast
      rcases hd with rfl | rfl <;> refine ⟨by omega, Or.inl ?_⟩ <;> simp
  apply Set.infinite_of_injective_forall_mem (f := fun n : ℕ => (⟨(v.1 + d * n, v.2), hray n⟩ :
    ((B : Set Pixel)? : Set Pixel)))
  · intro m n h
    simp only [Subtype.mk.injEq, Prod.mk.injEq] at h
    rcases hd with rfl | rfl <;> omega
  · intro n
    exact (SimpleGraph.ConnectedComponent.mem_supp_iff _ _).2
      (SimpleGraph.ConnectedComponent.eq.2 (hreach n).symm)
  · exact hfin


theorem ray_right (B : Finset Pixel) (v : Pixel) (h : ∀ b ∈ B, b.2 = v.2 → b.1 < v.1) :
    ∀ n : ℕ, (v.1 + 1 * n, v.2) ∉ B := by
  intro n hb
  have := h _ hb rfl
  simp at this; omega

theorem ray_left (B : Finset Pixel) (v : Pixel) (h : ∀ b ∈ B, b.2 = v.2 → v.1 < b.1) :
    ∀ n : ℕ, (v.1 + (-1) * n, v.2) ∉ B := by
  intro n hb
  have := h _ hb rfl
  simp at this; omega

/-- A pixel whose four 4-neighbours are all barrier is a finite component on its own. -/
```

```
theorem isolated (B : Finset Pixel) (h : Pixel) (hh : h ∉ B)
    (hn : ∀ u : Pixel, |h.1 - u.1| + |h.2 - u.2| = 1 → u ∈ B) :
    IsEnclosed ((complementGraph B).connectedComponentMk ⟨h, hh⟩) := by
  apply (Set.finite_singleton (⟨h, hh⟩ : ((B : Set Pixel)? : Set Pixel))).subset
  intro w hw
  rw [SimpleGraph.ConnectedComponent.mem_supp_iff, SimpleGraph.ConnectedComponent.eq] at hw
  obtain ⟨p⟩ := hw.symm
  cases p with
  | nil => rfl
  | @cons _ u _ hadj _ =>
    exfalso
    simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
      SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype] at hadj
    obtain ⟨-, hr | hr⟩ := hadj
    · exact u.2 (hn _ hr)
    · exact u.2 (hn _ (by rw [abs_sub_comm h.1, abs_sub_comm h.2]; exact hr))

theorem enclosed_eq_zero (B : Finset Pixel)
    (h : ∀ v (hv : v ∉ B), ¬ IsEnclosed ((complementGraph B).connectedComponentMk ⟨v, hv⟩)) :
    enclosed B = 0 := by
  rw [enclosed, Nat.card_eq_zero]
  left
  constructor
  rintro ⟨c, hc⟩
  induction c using SimpleGraph.ConnectedComponent.ind with
  | h v => exact h v.1 v.2 hc

theorem enclosed_eq_one (B : Finset Pixel) (h : Pixel) (hh : h ∉ B)
    (hiso : IsEnclosed ((complementGraph B).connectedComponentMk ⟨h, hh⟩))
    (hrest : ∀ v (hv : v ∉ B), v ≠ h →
      ¬ IsEnclosed ((complementGraph B).connectedComponentMk ⟨v, hv⟩)) :
    enclosed B = 1 := by
  rw [enclosed, Nat.card_eq_one_iff_unique]
  refine ⟨⟨?_⟩, ⟨⟨_, hiso⟩⟩⟩
  have key : ∀ x : {c : (complementGraph B).ConnectedComponent // IsEnclosed c},
      x.1 = (complementGraph B).connectedComponentMk ⟨h, hh⟩ := by
    rintro ⟨c, hc⟩
    induction c using SimpleGraph.ConnectedComponent.ind with
    | h v =>
      by_cases hvh : v.1 = h
      · obtain ⟨v, hv⟩ := v; subst hvh; rfl
      · exact absurd hc (hrest v.1 v.2 hvh)
  intro x y
  exact Subtype.ext ((key x).trans (key y).symm)

theorem enclosed_ring : enclosed ring = 1 := by
  refine enclosed_eq_one ring (1, 1) (by decide) (isolated _ _ _ ?_) ?_
  · rintro ⟨x, y⟩ h
    have h' : |1 - x| + |1 - y| = 1 := h
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq]
    rcases abs_cases (1 - x) with ⟨h1, h1'⟩ | ⟨h1, h1'⟩ <;>
    rcases abs_cases (1 - y) with ⟨h2, h2'⟩ | ⟨h2, h2'⟩ <;> omega
  · rintro ⟨x, y⟩ hv hne
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq, not_or] at hv
    simp only [ne_eq, Prod.mk.injEq] at hne
    by_cases hr : 2 < x ∨ y < 0 ∨ 2 < y
    · refine infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ ?_)
      simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega
    · refine infinite_of_ray _ _ _ (-1) (Or.inr rfl) (ray_left _ _ ?_)
      simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega

theorem enclosed_block : enclosed block = 0 := by
  refine enclosed_eq_zero block ?_
  rintro ⟨x, y⟩ hv
  simp only [block, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq, not_or] at hv
  by_cases hr : 1 < x ∨ y < 0 ∨ 1 < y
  · refine infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ ?_)
    simp only [block, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
    omega
  · refine infinite_of_ray _ _ _ (-1) (Or.inr rfl) (ray_left _ _ ?_)
    simp only [block, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
    omega

theorem enclosed_diamond : enclosed diamond = 1 := by
  refine enclosed_eq_one diamond (1, 1) (by decide) (isolated _ _ _ ?_) ?_
  · rintro ⟨x, y⟩ h
    have h' : |1 - x| + |1 - y| = 1 := h
    simp only [diamond, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq]
    rcases abs_cases (1 - x) with ⟨h1, h1'⟩ | ⟨h1, h1'⟩ <;>
    rcases abs_cases (1 - y) with ⟨h2, h2'⟩ | ⟨h2, h2'⟩ <;> omega
  · rintro ⟨x, y⟩ hv hne
    simp only [diamond, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq, not_or] at hv
    simp only [ne_eq, Prod.mk.injEq] at hne
    by_cases hr : 1 < x ∨ (x = 1 ∧ (y < 0 ∨ 2 < y))
```

```
    · refine infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ ?_)
      simp only [diamond, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega
    · refine infinite_of_ray _ _ _ (-1) (Or.inr rfl) (ray_left _ _ ?_)
      simp only [diamond, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega


theorem barrier_adj (B : Finset Pixel) (a b : Pixel) (ha : a ∈ B) (hb : b ∈ B)
    (h : a ≠ b ∧ |a.1 - b.1| ≤ 1 ∧ |a.2 - b.2| ≤ 1) :
    (barrierGraph B).Reachable ⟨a, ha⟩ ⟨b, hb⟩ := by
  refine SimpleGraph.Adj.reachable ?_
  simp only [barrierGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G8,
    SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype]
  exact ⟨h.1, Or.inl h.2⟩

theorem C8_eq_one (B : Finset Pixel) (b₀ : Pixel) (h₀ : b₀ ∈ B)
    (hr : ∀ b (hb : b ∈ B), (barrierGraph B).Reachable ⟨b₀, h₀⟩ ⟨b, hb⟩) : C8 B = 1 := by
  rw [C8, Nat.card_eq_one_iff_unique]
  refine ⟨⟨fun x y => ?_⟩, ⟨(barrierGraph B).connectedComponentMk ⟨b₀, h₀⟩⟩⟩
  induction x using SimpleGraph.ConnectedComponent.ind with
  | h v =>
  induction y using SimpleGraph.ConnectedComponent.ind with
  | h w =>
    exact SimpleGraph.ConnectedComponent.eq.2 ((hr v.1 v.2).symm.trans (hr w.1 w.2))

theorem C8_empty_eq : C8 ∅ = 0 := by
  rw [C8, Nat.card_eq_zero]
  left
  constructor
  intro c
  induction c using SimpleGraph.ConnectedComponent.ind with
  | h v => simpa using v.2

theorem enclosed_empty : enclosed ∅ = 0 :=
  enclosed_eq_zero ∅ fun v _ => infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ (by simp))

theorem C8_block : C8 block = 1 := by
  refine C8_eq_one block (0, 0) (by decide) ?_
  intro b hb
  simp only [block, Finset.mem_insert, Finset.mem_singleton] at hb
  rcases hb with rfl | rfl | rfl | rfl
  · rfl
  all_goals exact barrier_adj _ _ _ _ _ (by decide)

theorem C8_diamond : C8 diamond = 1 := by
  refine C8_eq_one diamond (1, 0) (by decide) ?_
  intro b hb
  simp only [diamond, Finset.mem_insert, Finset.mem_singleton] at hb
  rcases hb with rfl | rfl | rfl | rfl
  · rfl
  · exact barrier_adj _ _ _ _ _ (by decide)
  · exact barrier_adj _ _ _ _ _ (by decide)
  · exact (barrier_adj _ (1, 0) (0, 1) (by decide) (by decide) (by decide)).trans
      (barrier_adj _ _ _ _ _ (by decide))

theorem C8_ring : C8 ring = 1 := by
  refine C8_eq_one ring (1, 0) (by decide) ?_
  have h01 : (0, 1) ∈ ring := by decide
  have h21 : (2, 1) ∈ ring := by decide
  intro b hb
  simp only [ring, Finset.mem_insert, Finset.mem_singleton] at hb
  rcases hb with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
  · exact barrier_adj _ _ _ _ _ (by decide)
  · rfl
  · exact barrier_adj _ _ _ _ _ (by decide)
  · exact barrier_adj _ _ _ _ _ (by decide)
  · exact barrier_adj _ _ _ _ _ (by decide)
  · exact (barrier_adj _ (1, 0) (0, 1) (by decide) h01 (by decide)).trans
      (barrier_adj _ _ _ _ _ (by decide))
  · exact (barrier_adj _ (1, 0) (0, 1) (by decide) h01 (by decide)).trans
      (barrier_adj _ _ _ _ _ (by decide))
  · exact (barrier_adj _ (1, 0) (2, 1) (by decide) h21 (by decide)).trans
      (barrier_adj _ _ _ _ _ (by decide))

end Controls

open Controls in
/-- Witness, semantic: the ring has one enclosed component, and the identity
`enclosed = C8 − χ` holds for it (`1 = 1 − 0`). -/
example : enclosed ring = 1 ∧ C8 ring = 1 ∧ (enclosed ring : ℤ) = C8 ring - euler ring := by
  refine ⟨enclosed_ring, C8_ring, ?_⟩
  rw [enclosed_ring, C8_ring]
  decide

open Controls in
```

```
/-- Negative control 1, semantic: the block encloses nothing (‘0 = 1 − 1‘). -/
example : enclosed block = 0 ∧ C8 block = 1 ∧ (enclosed block : ℤ) = C8 block - euler block := by
  refine ⟨enclosed_block, C8_block, ?_⟩
  rw [enclosed_block, C8_block]
  decide

open Controls in
/-- Negative control 2, semantic: the diamond encloses its centre (‘1 = 1 − 0‘). -/
example : enclosed diamond = 1 ∧ C8 diamond = 1 ∧
    (enclosed diamond : ℤ) = C8 diamond - euler diamond := by
  refine ⟨enclosed_diamond, C8_diamond, ?_⟩
  rw [enclosed_diamond, C8_diamond]
  decide

open Controls in
/-- Empty raster, semantic: ‘C8 = 0‘, ‘χ = 0‘, nothing enclosed. -/
example : enclosed ∅ = 0 ∧ C8 ∅ = 0 ∧ euler ∅ = 0 :=
  ⟨enclosed_empty, C8_empty_eq, by decide⟩

end ViridisStatements.P1.Insertion
```

Other checks

- Lean toolchain leanprover/lean4:v4.28.0, Mathlib 8f9d9cff6bd7; kernel replay by `leanchecker`.
- References: 7 entries, 7 DOIs, 7 resolved, 0 unresolved, 0 unchecked.
- Agent paper: compiled.
- Statement proposals: 0.

How to read this record

ASSUMED: not verified (the default). **PROVEN**: a Lean 4 proof of a statement sealed before the attempt, compiled with the pinned toolchain, matched definitionally to the sealed statement, using only the axioms propext, Classical.choice and Quot.sound, and replayed through the kernel. **TESTED/REPLICATED**: reproduced by the verifier (not yet enabled). **ENDORSED/ADOPTED**: granted by people. Novelty is never a label. **TRIVIAL** marks a sealed statement that one of the engine's probe tactics already closes.

This paper was drafted by an AI research agent (Claude, by Anthropic) running the Viridis research engine; formal proof search may use Aristotle (Harmonic). Justin Hart directs the research program. The labels on this page were assigned by an automated verifier, not by the agent, and only they carry certification. Claims in the paper that are not listed here as PROVEN are ASSUMED.

Reproduce everything from `REPRODUCE.md` in the run folder; the run's files are listed with hashes in `MANIFEST.sha256`.

A machine-checked raster bridge for *Trees Cannot Cut*: single-pixel insertion preserves enc $- C_8 + \chi$

Justin Hart
Viridis LLC

9 November 2026

Abstract

For a finite set $B \subset \mathbb{Z}^2$ of barrier pixels, read with 8-connected barriers and a 4-connected complement, the expected discrete form of Alexander duality is the identity $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$, where $\mathrm{enc}(B)$ is the number of finite 4-components of $\mathbb{Z}^2 \setminus B$, C_8 counts 8-components of B and $\chi(K(B)) = V - E + F$ is the Euler characteristic of the union of the closed pixel squares. This identity would replace the topological hypothesis `FaceCount` that the machine-checked core of *Trees Cannot Cut* currently assumes. The project climbs a three-rung ladder. For the first rung (L1), Gray's bit-quad formula $4\chi = Q_1 - Q_3 - 2Q_D$, we gave a Lean 4 proof in an earlier session by double counting over the vertices of $K(B)$. This version adds precise Lean statements proposed for sealing for the other two rungs: `P1.Insertion` (L2: inserting any pixel leaves enclosed $- C_8 + \chi$ unchanged, with no hypothesis on its 3×3 neighbourhood) and `P1.Bridge` (L3: the identity above, together with finiteness of the set of enclosed components). Both use Mathlib's graph library on all of $\mathbb{Z}^2$ and define a component as enclosed exactly when it is finite. Both restate χ verbatim from the sealed L1 module and import nothing but Mathlib. Each module checks the project's controls (the 3×3 ring, the solid 2×2 block, and the four-pixel diamond that is enclosing only under (8,4) connectivity) by `decide` on a computable bounded-box model. It also gives Lean proofs of the same control values for the non-computable definitions that the statements actually use. A second, bit-level model lets `decide` run through all 256 configurations of the eight neighbours of an inserted pixel, both alone and inside a closed ring. In every case the insertion leaves enclosed $- C_8 + \chi$ at 0. An independent brute-force computation finds no counterexample to L3 on all 65,536 subsets of a 4×4 box or on 3,000 random rasters, and none to L2 across all 256 neighbourhood configurations. Both statements have since been sealed. The present revision gives a Lean proof of L2. Earlier sessions gave Lean proofs that the change in χ is local, that C_8 changes by $1 - k$ (with k the number of 8-components of B touching the inserted pixel p), that every finite raster has finitely many enclosed components, and of the change in the enclosed count. Combined with an induction in raster-scan order, they reduced L2 to a half-plane digital Jordan lemma about the pixel S below p, and proved one direction of it by crossing parity. We prove the other direction by following the crack boundary of the 8-component of the lower-right neighbour SE. The (8,4) crack-successor map has an explicit left inverse, so it permutes the finite set of cracks. Injectivity alone then forces the orbit from S to reach the pixel E to the right of p without passing through p. As a corollary, we give a Lean proof of the bridge equation $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$ for every finite raster. The proof uses no planar topology beyond these two discrete arguments.

1 Introduction

Hart's *Trees Cannot Cut* [3] argues that fuel breaks and habitat corridors split a forest landscape into separate compartments only through their cycles: an acyclic corridor leaves the fuel matrix

connected. The graph theory behind this claim is machine-checked, but the step from topology to graph theory is carried as an explicit hypothesis (`FaceCount`), which says that the number of compartments equals the circuit rank plus one. The manuscript justifies it by Alexander duality. Mathlib [7] has neither Alexander duality nor a planar Euler formula in a form that applies here.

Fire and fuel-break data usually come as rasters, so project P1 aims to discharge the hypothesis in the raster setting. For a finite barrier raster B with 8-connected barriers and a 4-connected complement, the target identity is

$$\#\{\text{enclosed 4-components of } \mathbb{Z}^2 \setminus B\} \;=\; C_8(B) - \chi(K(B)).$$

The proposed route is a ladder of three lemmas. L1 is a local formula for $\chi(K(B))$. L2 says that adding one pixel preserves enclosed $- C_8 + \chi$. L3 is the bridge itself, by induction on $|B|$. The first version of this paper did L1: it gives a Lean proof of Gray's bit-quad formula in its 8-connected form (Sections 3–7). This version adds exact Lean statements, proposed for sealing, for L2 and L3 (Section 8). It also adds machine-checked sanity checks on their definitions and computational evidence that the proposed statements are true (Section 11). The 2026-11-04 revision made the proposal modules self-contained (Section 8.1) and added an exhaustive kernel check of all 256 neighbourhood configurations of L2 (Section 8.4). A proposed statement is a target, not a result: those sections do not prove L2 or L3. L1 matters for the rest of the ladder because it makes the change in χ under a single-pixel insertion a purely local quantity: only the four windows around the new pixel change.

Both proposed statements were later sealed unchanged. The 2026-11-08 revision (Section 9) reduced L2 to one direction of a half-plane digital Jordan lemma about the pixel below the inserted one. It proved the other direction, and with it half of the bridge identity, enc $\geq C_8 - \chi$. The present revision closes the gap (Section 10). We give a Lean proof of the sealed statement `P1.Insertion` and, as a corollary, of the bridge equation enc $= C_8 - \chi$ for every finite raster. This is the equation of the sealed L3 statement `P1.Bridge`. We do not claim that statement here, since it is a separate task and has its own exact form.

2 Background and related work

Gray [2] introduced locally countable properties of binary images, among them area, perimeter and the Euler number. He showed that the Euler number can be computed from counts of 2×2 "bit-quad" patterns, with different weightings for 4- and 8-connectivity of the foreground. The (8,4) pairing of connectivity for foreground and background goes back to Rosenfeld [5], who showed that with this pairing a simple closed digital curve separates the plane into exactly two components. Its role in digital Jordan-type theorems is surveyed by Kong and Rosenfeld [4]. In the 8-connected reading, two pixels that share only a corner touch in $K(B)$. The two diagonal patterns then contribute -2 to 4χ, because their shared corner is one vertex where four edges meet. Our Lean development uses Lean 4 [1] and Mathlib [7]. We are not aware of an earlier machine-checked proof of the bit-quad formula, but we have not searched exhaustively and make no claim of novelty.

3 Setting: the sealed statement

The statement module `ViridisStatements.P1.BitQuad` fixes the following definitions. A pixel $p = (i, j) \in \mathbb{Z}^2$ stands for the closed square $[i, i+1] \times [j, j+1]$.

- vertices$(B) = \bigcup_{p \in B} \text{corners}(p)$, where corners$(i, j) = \{(i, j), (i+1, j), (i, j+1), (i+1, j+1)\}$.

- $\mathrm{hEdges}(B) = \bigcup_{(i,j)\in B}\{(i,j),(i,j{+}1)\}$ (horizontal edges named by their left end) and $\mathrm{vEdges}(B) = \bigcup_{(i,j)\in B}\{(i,j),(i+1,j)\}$ (vertical edges named by their lower end).
- The Euler characteristic, computed in $\mathbb{Z}$:

$$\chi(B) = |\mathrm{vertices}(B)| - (|\mathrm{hEdges}(B)| + |\mathrm{vEdges}(B)|) + |B|.$$

- The window at a lattice point $v = (x, y)$ consists of four bits, $a = [(x-1, y-1) \in B]$, $b = [(x, y-1) \in B]$, $c = [(x-1, y) \in B]$ and $d = [(x, y) \in B]$, with $n(v) = a + b + c + d$.
- Q_1, Q_3 and Q_D count the vertices v with $n(v) = 1$, with $n(v) = 3$, and with $\{a, d\}$ or $\{b, c\}$ as the only set bits.

Every window that meets B is centred on a vertex of $K(B)$, so counting over the vertices counts every such window exactly once.

Theorem 1 (`P1.BitQuad`)**.** *For every finite $B \subset \mathbb{Z}^2$, $4\,\chi(K(B)) = Q_1(B) - Q_3(B) - 2\,Q_D(B)$.*

4 Proof architecture

Write $\mathcal{V} = \mathrm{vertices}(B)$, and use $[P] \in \{0, 1\}$ for the indicator of a proposition P.

Lemma 2 (Edge membership)**.** $(i,j) \in \mathrm{hEdges}(B) \iff (i,j) \in B \vee (i,j-1) \in B$*, and* $(i,j) \in \mathrm{vEdges}(B) \iff (i,j) \in B \vee (i-1,j) \in B$*.*

Lemma 3 (Shift counting)**.** *Let $S, \mathcal{V} \subset \mathbb{Z}^2$ be finite and $s \in \mathbb{Z}^2$. If $x + s \in \mathcal{V}$ for every $x \in S$, then $\sum_{v\in\mathcal{V}} [v - s \in S] = |S|$.*

Proof. The map $v \mapsto v - s$ is a bijection from $\{v \in \mathcal{V} : v - s \in S\}$ onto S. Its inverse is $x \mapsto x + s$, which lands in $\mathcal{V}$ by hypothesis. □

Lemma 3 does all of the double counting. Every pixel $x \in B$ has its four corners x, $x + (1, 0)$, $x + (0, 1)$ and $x + (1, 1)$ in $\mathcal{V}$. Applying the lemma with the four shifts gives

$$4F = \sum_{v\in\mathcal{V}} n(v).$$

By Lemma 2, both ends x and $x + (1, 0)$ of a horizontal edge are corners of the pixel above or below it, and likewise for vertical edges. Write the arms of v as left $[v - (1, 0) \in \mathrm{hEdges}]$, right $[v \in \mathrm{hEdges}]$, down $[v - (0, 1) \in \mathrm{vEdges}]$ and up $[v \in \mathrm{vEdges}]$. Applying the lemma once more for each arm gives

$$2E = \sum_{v\in\mathcal{V}} \mathrm{arms}(v), \qquad V = \sum_{v\in\mathcal{V}} 1.$$

Hence $4\chi = \sum_{v\in\mathcal{V}} \big(4 - 2\,\mathrm{arms}(v) + n(v)\big)$. Lemma 2 expresses each arm through the window bits. The left arm is $a \vee c$, the right arm $b \vee d$, the down arm $a \vee b$ and the up arm $c \vee d$. The summand therefore depends only on (a, b, c, d), and Table 1 evaluates it. Every vertex is a corner of some pixel, which must be one of the four window cells, so the empty pattern never occurs. On the other 15 patterns the summand equals $[n = 1] - [n = 3] - 2[\mathrm{diagonal}]$. Summing over $\mathcal{V}$ proves Theorem 1.

Pattern class	#patterns	n	arms	$4 - 2\,\text{arms} + n$
empty (cannot occur)	1	0	0	4
one pixel	4	1	2	1
adjacent pair	4	2	3	0
diagonal pair	2	2	4	-2
three pixels	4	3	4	-1
full	1	4	4	0

Table 1: The local summand on the 16 window patterns.

5 The Lean development

The candidate file `proofs/P1.BitQuad.lean` (145 lines) follows the architecture above.

- `mem_vertices`, `mem_corners`, `mem_hEdges` and `mem_vEdges` unfold the membership predicates (Lemma 2).
- `card_shift` proves Lemma 3 with `Finset.card_bij`. Its corollary `sum_shift` restates it as a sum of $\{0,1\}$-valued integer indicators via `Finset.sum_boole`.
- In `main`, the eight instances of `sum_shift` (four for faces, two for each edge direction) and the three expressions of Q_1, Q_3 and Q_D as indicator sums reduce the goal to a single `Finset.sum_congr`. The pointwise goal is closed by `by_cases` on the four window memberships followed by `simp`, which also uses the fact that the window at a vertex is non-empty.

The brief suggested evaluating the 16 patterns with `decide`. That proved unnecessary: once the window memberships are case-split, `simp` closes all 16 branches directly. The file uses no `decide` on large objects and no `native_decide`. The project checker (`viridis-lean-check`) reports that the file compiles under Lean `v4.28.0` with the pinned Mathlib, matches the sealed statement, depends only on `propext`, `Classical.choice` and `Quot.sound`, and replays through the kernel. Whether the claim is accepted is decided by the project's independent verifier; see its certification page. The complete file follows.

```
import Mathlib
import ViridisStatements.P1.BitQuad

namespace Candidate

open ViridisStatements.P1.BitQuad

/-- Membership in ‘vertices‘. -/
theorem mem_vertices (B : Finset Pixel) (v : ℤ × ℤ) :
    v ∈ vertices B ↔ ∃ p ∈ B, v ∈ corners p := by
  unfold vertices
  simp only [Finset.mem_biUnion]

theorem mem_corners (p v : ℤ × ℤ) :
    v ∈ corners p ↔ v = (p.1, p.2) ∨ v = (p.1 + 1, p.2) ∨ v = (p.1, p.2 + 1) ∨
      v = (p.1 + 1, p.2 + 1) := by
  unfold corners
  simp only [Finset.mem_insert, Finset.mem_singleton]

/-- A horizontal edge is present iff the pixel above or below it is in ‘B‘. -/
```

```
theorem mem_hEdges (B : Finset Pixel) (h : ℤ × ℤ) :
    h ∈ hEdges B ↔ (h.1, h.2) ∈ B ∨ (h.1, h.2 - 1) ∈ B := by
  unfold hEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- A vertical edge is present iff the pixel right or left of it is in `B`. -/
theorem mem_vEdges (B : Finset Pixel) (e : ℤ × ℤ) :
    e ∈ vEdges B ↔ (e.1, e.2) ∈ B ∨ (e.1 - 1, e.2) ∈ B := by
  unfold vEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- Shift-counting: if every translate `x + (a, b)` of `S` lies in `V`, then
counting the `v ∈ V` with `v - (a, b) ∈ S` gives `|S|`. -/
theorem card_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (V.filter fun v => (v.1 - a, v.2 - b) ∈ S).card = S.card := by
  apply Finset.card_bij (fun v _ => (v.1 - a, v.2 - b))
  · intro v hv
    exact (Finset.mem_filter.mp hv).2
  · intro v₁ _ v₂ _ h
    simp only [Prod.mk.injEq] at h
    ext <;> omega
  · intro x hx
    refine ⟨(x.1 + a, x.2 + b), Finset.mem_filter.mpr ⟨hS x hx, ?_⟩, ?_⟩
    · simpa using hx
    · simp

theorem sum_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (∑ v ∈ V, if (v.1 - a, v.2 - b) ∈ S then (1 : ℤ) else 0) = S.card := by
  rw [Finset.sum_boole, card_shift V S a b hS]

theorem main : ViridisStatements.P1.BitQuad.Statement := by
  intro B
  set V := vertices B with hV
  -- inclusion facts
  have hB : ∀ x ∈ B, ∀ a b : ℤ, (a = 0 ∨ a = 1) → (b = 0 ∨ b = 1) →
      (x.1 + a, x.2 + b) ∈ V := by
    intro x hx a b ha hb
    rw [hV, mem_vertices]
    refine ⟨x, hx, ?_⟩
    rw [mem_corners]
    rcases ha with rfl | rfl <;> rcases hb with rfl | rfl <;> simp
  have hH : ∀ x ∈ hEdges B, ∀ a : ℤ, (a = 0 ∨ a = 1) → (x.1 + a, x.2 + 0) ∈ V := by
    intro x hx a ha
```

```
  rw [mem_hEdges] at hx
  rcases hx with hx | hx
  · have := hB _ hx a 0 ha (Or.inl rfl)
    simpa using this
  · have := hB _ hx a 1 ha (Or.inr rfl)
    simpa using this
have hVe : ∀ x ∈ vEdges B, ∀ b : ℤ, (b = 0 ∨ b = 1) → (x.1 + 0, x.2 + b) ∈ V := by
  intro x hx b hb
  rw [mem_vEdges] at hx
  rcases hx with hx | hx
  · have := hB _ hx 0 b (Or.inl rfl) hb
    simpa using this
  · have := hB _ hx 1 b (Or.inr rfl) hb
    simpa using this
-- the four expressions as sums over V
have eV : ((V.card : ℤ)) = ∑ v ∈ V, (1 : ℤ) := by simp
have eF11 := sum_shift V B 1 1 (fun x hx => hB x hx 1 1 (Or.inr rfl) (Or.inr rfl))
have eF01 := sum_shift V B 0 1 (fun x hx => hB x hx 0 1 (Or.inl rfl) (Or.inr rfl))
have eF10 := sum_shift V B 1 0 (fun x hx => hB x hx 1 0 (Or.inr rfl) (Or.inl rfl))
have eF00 := sum_shift V B 0 0 (fun x hx => hB x hx 0 0 (Or.inl rfl) (Or.inl rfl))
have eH10 := sum_shift V (hEdges B) 1 0 (fun x hx => hH x hx 1 (Or.inr rfl))
have eH00 := sum_shift V (hEdges B) 0 0 (fun x hx => hH x hx 0 (Or.inl rfl))
have eV01 := sum_shift V (vEdges B) 0 1 (fun x hx => hVe x hx 1 (Or.inr rfl))
have eV00 := sum_shift V (vEdges B) 0 0 (fun x hx => hVe x hx 0 (Or.inl rfl))
have eQ1 : ((Q1 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 1 then (1 : ℤ) else 0 := by
  rw [Finset.sum_boole]; rfl
have eQ3 : ((Q3 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 3 then (1 : ℤ) else 0 := by
  rw [Finset.sum_boole]; rfl
have eQD : ((QD B : ℕ) : ℤ) = ∑ v ∈ V, if isDiagonal B v = true then (1 : ℤ) else 0 := by
  rw [Finset.sum_boole]; rfl
have key : 4 * euler B = ∑ v ∈ V,
    ((4 : ℤ) - 2 * ((if (v.1 - 1, v.2 - 0) ∈ hEdges B then 1 else 0)
      + (if (v.1 - 0, v.2 - 0) ∈ hEdges B then 1 else 0)
      + (if (v.1 - 0, v.2 - 1) ∈ vEdges B then 1 else 0)
      + (if (v.1 - 0, v.2 - 0) ∈ vEdges B then 1 else 0))
      + ((if (v.1 - 1, v.2 - 1) ∈ B then 1 else 0)
      + (if (v.1 - 0, v.2 - 1) ∈ B then 1 else 0)
      + (if (v.1 - 1, v.2 - 0) ∈ B then 1 else 0)
      + (if (v.1 - 0, v.2 - 0) ∈ B then 1 else 0))) := by
  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.mul_sum]
  rw [eF11, eF01, eF10, eF00, eH10, eH00, eV01, eV00]
  unfold euler
  rw [← hV]
  simp
  ring
rw [key, eQ1, eQ3, eQD, ← Finset.sum_sub_distrib, Finset.mul_sum,
  ← Finset.sum_sub_distrib]
apply Finset.sum_congr rfl
intro v hv
obtain ⟨x, y⟩ := v
have hne : (x - 1, y - 1) ∈ B ∨ (x, y - 1) ∈ B ∨ (x - 1, y) ∈ B ∨ (x, y) ∈ B := by
  rw [hV, mem_vertices] at hv
  obtain ⟨p, hp, hc⟩ := hv
  rw [mem_corners] at hc
  rcases hc with h | h | h | h <;> simp only [Prod.mk.injEq] at h <;> obtain ⟨rfl, rfl⟩ := h
  · right; right; right; exact hp
  · right; right; left; simpa using hp
  · right; left; simpa using hp
  · left; simpa using hp
```

```
  unfold windowCount isDiagonal bits
  simp only [sub_zero, mem_hEdges, mem_vEdges]
  by_cases ha : (x - 1, y - 1) ∈ B <;> by_cases hb : (x, y - 1) ∈ B <;>
    by_cases hc : (x - 1, y) ∈ B <;> by_cases hd : (x, y) ∈ B <;>
    simp [ha, hb, hc, hd] at hne ⊢

end Candidate
```

6 Results

- We give a Lean proof of Theorem 1, the sealed statement `P1.BitQuad`, for all finite rasters.
- As an independent computational check, which is evidence and not a proof, the script `code/bitquad_check.py` recomputes both sides for all $2^{16} = 65{,}536$ subsets of a 4×4 box. It finds no counterexample (`results/bitquad_4x4.json`). Euler characteristics from -1 to 4 occur in that box.

7 How L1 serves the L2/L3 bridge

Let $p \notin B$ and $B' = B \cup \{p\}$. The windows whose bits differ between B and B' are exactly the four windows centred on the corners of p. By Theorem 1 applied to B and to B', $4(\chi(B') - \chi(B))$ equals the change in $Q_1 - Q_3 - 2Q_D$ over those four windows. That change depends only on the 3×3 neighbourhood of p. This is the "local half" of L2. The non-local half, the change in C_8 and in the number of enclosed 4-components of the complement, still needs a separate argument: a pixel joining two 8-components can close off a new bounded region. L1 contributes nothing there. The negative controls listed in the project context (the solid 2×2 block, whose 8-neighbour graph K_4 has circuit rank 3 yet encloses nothing, and the four-pixel N/S/E/W cross enclosing its centre) show that L2 must be stated in terms of $\chi(K(B))$ and not in terms of the graph circuit rank.

8 Proposed statements for L2 and L3

The two modules `proposals/P1.Insertion.lean` and `proposals/P1.Bridge.lean` are *proposals*. They become targets only if they are sealed, and nothing in this section is a proof of either statement. Proposals may not import one another, so the two modules carry identical copies of the connectivity definitions.

8.1 Self-contained modules

The first draft of these modules (2026-11-03) imported the sealed L1 module `ViridisStatements.P1.BitQuad` for χ. In the verifier's proposal environment that import did not resolve: the compiled object file for the module was missing. Both drafts were therefore rejected before any probe ran. The brief anticipates this: if the L1 module cannot be imported, its definitions are restated identically. Both modules now import only Mathlib. Each restates `Pixel`, `corners`, `vertices`, `hEdges`, `vEdges` and `euler` character for character from the sealed L1 module, and a textual `diff` confirms that the copies are identical. The two `Statement` bodies are unchanged from the first draft. If the import becomes available, the restated block can be replaced by `open ViridisStatements.P1.BitQuad` without any change in meaning.

8.2 Definitions and conventions

Definition 4. Let G_4 and G_8 be the graphs on $\mathbb{Z}^2$ in which distinct p, q are adjacent when $|p_1 - q_1| + |p_2 - q_2| = 1$, respectively when $|p_1 - q_1| \le 1$ and $|p_2 - q_2| \le 1$. For a finite B, $C_8(B)$ is the number of connected components of the subgraph of G_8 induced on B. A connected component of the subgraph of G_4 induced on $\mathbb{Z}^2 \setminus B$ is *enclosed* when it is finite, and $\mathrm{enc}(B)$ is the number of enclosed components.

The brief for this task fixes the exterior convention, and the definitions follow it literally. The complement is taken in all of $\mathbb{Z}^2$, never inside a cropping window. "Enclosed" means "finite", so no bounding box enters the statements. In Lean the graphs are built with `SimpleGraph.fromRel`, which symmetrises a relation and removes loops, and components are Mathlib's `ConnectedComponent` of an induced graph (Listing 1).

Listing 1: Definitions and the two proposed statements (each Statement lives in its own module).

```
def G4 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| + |p.2 - q.2| = 1
def G8 : SimpleGraph Pixel :=
  SimpleGraph.fromRel fun p q => |p.1 - q.1| ≤ 1 ∧ |p.2 - q.2| ≤ 1
def barrierGraph (B : Finset Pixel) : SimpleGraph (B : Set Pixel) :=
  G8.induce (B : Set Pixel)
def complementGraph (B : Finset Pixel) : SimpleGraph ((B : Set Pixel)ᶜ : Set Pixel) :=
  G4.induce (B : Set Pixel)ᶜ
noncomputable def C8 (B : Finset Pixel) : ℕ :=
  Nat.card (barrierGraph B).ConnectedComponent
def IsEnclosed {B : Finset Pixel} (c : (complementGraph B).ConnectedComponent) : Prop :=
  c.supp.Finite
noncomputable def enclosed (B : Finset Pixel) : ℕ :=
  Nat.card {c : (complementGraph B).ConnectedComponent // IsEnclosed c}

-- P1.Bridge (L3)
def Statement : Prop :=
  ∀ B : Finset Pixel,
    {c : (complementGraph B).ConnectedComponent | IsEnclosed c}.Finite ∧
      (enclosed B : ℤ) = C8 B - euler B

-- P1.Insertion (L2)
noncomputable def invariant (B : Finset Pixel) : ℤ :=
  (enclosed B : ℤ) - C8 B + euler B
def Statement : Prop :=
  ∀ (B : Finset Pixel) (p : Pixel), p ∉ B → invariant (insert p B) = invariant B
```

L3 (`P1.Bridge`). For every finite B, the set of enclosed components is finite and $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$ in $\mathbb{Z}$. The finiteness clause is deliberate. `Nat.card` returns 0 on an infinite type, so without the clause the equation could be met in a degenerate way. With it, the count means what it says. The clause is true because every component outside the bounding box of B contains a ray.

L2 (`P1.Insertion`). For every finite B and every $p \notin B$, $\mathrm{enc} - C_8 + \chi$ takes the same value on $B \cup \{p\}$ and on B. The statement puts no condition on the 3×3 neighbourhood of p. All 256 configurations of its eight neighbours are therefore covered, in every global context. These include insertions that merge several 8-components, insertions that close off a hole, and insertions that fill

one. By L1, $\Delta\chi$ is a function of the neighbourhood (Section 7). ΔC_8 and Δenc are not local, and they are where the proof effort lies.

Relation between the rungs. L3's equation follows from L2 by induction on $|B|$ from the empty raster, where both sides are 0. Conversely, L3's equation gives L2 at once. The two rungs are equivalent given the base case. The split organises the proof around one inductive step; it does not grade the statements by strength.

8.3 Controls

The definitions of C_8 and enc are not decidable, since they quantify over infinite components of $\mathbb{Z}^2$. Each module therefore also contains a computable *bounded-box model*. It flood-fills the complement inside the bounding box of $B \cup \{(0,0)\}$ with a one-pixel margin, starting from the box corner, and counts components by their lexicographically least pixel. The statements do not mention this model, and we do not prove that it agrees with the definitions. Against the box model, `decide` checks the values in Table 2. The `P1.Insertion` module also checks four non-local insertions: closing the ring's top edge, merging the ring's two columns through a corner, completing the diamond, and filling the ring's centre.

raster	C_8	χ	enc	role
empty	0	0	0	base case
3×3 ring	1	0	1	witness
solid 2×2 block	1	1	0	negative control 1 (K_4 has circuit rank 3)
N/S/E/W diamond	1	0	1	negative control 2 (4-adjacency graph has no edges)
diagonal pair	1	1	0	corner contact

Table 2: Control values. All rows are checked by `decide` on the box model. For the first four rows we also give Lean proofs for the semantic definitions C_8 and enc themselves.

To connect the controls to the definitions that the statements actually use, each module proves the control values for C_8 and enc directly. Two general lemmas carry these proofs (Listing 2). If a horizontal ray from a non-barrier pixel avoids B, the pixel's component is infinite (`infinite_of_ray`). A non-barrier pixel whose four 4-neighbours lie in B forms a finite component (`isolated`). For the ring, every non-barrier pixel other than the centre has a free ray to the left or right. The centre is isolated. Hence exactly one component is enclosed. The block and diamond are handled the same way, and $C_8 = 1$ follows from explicit 8-paths. Combined with `decide` on χ, this checks the identity enc $= C_8 - \chi$ for the ring ($1 = 1 - 0$), the block ($0 = 1 - 1$) and the diamond ($1 = 1 - 0$) on the semantic definitions.

Listing 2: Key lemmas behind the semantic controls (excerpt from the proposal modules).

```
/-- A horizontal ray of non-barrier pixels makes a component infinite. -/
theorem infinite_of_ray (B : Finset Pixel) (v : Pixel) (hv : v ∉ B) (d : ℤ) (hd : d = 1 ∨ d = -1)
    (hray : ∀ n : ℕ, (v.1 + d * n, v.2) ∉ B) :
    ¬ IsEnclosed ((complementGraph B).connectedComponentMk ⟨v, hv⟩) := by
  intro hfin
  have hreach : ∀ n : ℕ, (complementGraph B).Reachable ⟨v, hv⟩ ⟨(v.1 + d * n, v.2), hray n⟩ := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
```

```
      refine ih.trans (SimpleGraph.Adj.reachable ?_)
      simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
        SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype, ne_eq, Prod.mk.injEq]
      push_cast
      rcases hd with rfl | rfl <;> refine ⟨by omega, Or.inl ?_⟩ <;> simp
  apply Set.infinite_of_injective_forall_mem (f := fun n : ℕ => (⟨(v.1 + d * n, v.2), hray n⟩ :
    ((B : Set Pixel)ᶜ : Set Pixel)))
  · intro m n h
    simp only [Subtype.mk.injEq, Prod.mk.injEq] at h
    rcases hd with rfl | rfl <;> omega
  · intro n
    exact (SimpleGraph.ConnectedComponent.mem_supp_iff _ _).2
      (SimpleGraph.ConnectedComponent.eq.2 (hreach n).symm)
  · exact hfin


/-- A pixel whose four 4-neighbours are all barrier is a finite component on its own. -/
theorem isolated (B : Finset Pixel) (h : Pixel) (hh : h ∉ B)
    (hn : ∀ u : Pixel, |h.1 - u.1| + |h.2 - u.2| = 1 → u ∈ B) :
    IsEnclosed ((complementGraph B).connectedComponentMk ⟨h, hh⟩) := by
  apply (Set.finite_singleton (⟨h, hh⟩ : ((B : Set Pixel)ᶜ : Set Pixel))).subset
  intro w hw
  rw [SimpleGraph.ConnectedComponent.mem_supp_iff, SimpleGraph.ConnectedComponent.eq] at hw
  obtain ⟨p⟩ := hw.symm
  cases p with
  | nil => rfl
  | @cons _ u _ hadj _ =>
    exfalso
    simp only [complementGraph, SimpleGraph.comap_adj, SimpleGraph.induce, G4,
      SimpleGraph.fromRel_adj, Function.Embedding.coe_subtype] at hadj
    obtain ⟨-, hr | hr⟩ := hadj
    · exact u.2 (hn _ hr)
    · exact u.2 (hn _ (by rw [abs_sub_comm h.1, abs_sub_comm h.2]; exact hr))

theorem enclosed_ring : enclosed ring = 1 := by
  refine enclosed_eq_one ring (1, 1) (by decide) (isolated _ _ _ ?_) ?_
  · rintro ⟨x, y⟩ h
    have h' : |1 - x| + |1 - y| = 1 := h
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq]
    rcases abs_cases (1 - x) with ⟨h1, h1'⟩ | ⟨h1, h1'⟩ <;>
    rcases abs_cases (1 - y) with ⟨h2, h2'⟩ | ⟨h2, h2'⟩ <;> omega
  · rintro ⟨x, y⟩ hv hne
    simp only [ring, Finset.mem_insert, Finset.mem_singleton, Prod.mk.injEq, not_or] at hv
    simp only [ne_eq, Prod.mk.injEq] at hne
    by_cases hr : 2 < x ∨ y < 0 ∨ 2 < y
    · refine infinite_of_ray _ _ _ 1 (Or.inl rfl) (ray_right _ _ ?_)
      simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega
    · refine infinite_of_ray _ _ _ (-1) (Or.inr rfl) (ray_left _ _ ?_)
      simp only [ring, Finset.mem_insert, Finset.mem_singleton, forall_eq_or_imp, forall_eq]
      omega
```

8.4 All 256 neighbourhood configurations by `decide`

The task brief asks that L2 account for every configuration of the 3×3 neighbourhood. The statement itself does so by having no hypothesis on the neighbourhood. The first draft checked only four hand-picked insertions in Lean, because the `Finset` flood fill of the box model is too slow

for the kernel to run through all 256 cases. This revision adds a second test oracle (namespace `Mask` in both modules). A 7×7 box is a 49-bit natural number with pixel (x, y) at bit $7y + x$. Neighbourhoods are computed with shifts and masks, components by iterated flood fill from the lowest set bit, and $V - E + F$ by counting the bits of shifted unions. These use the same vertex and edge naming as `euler`. The Lean kernel evaluates these natural-number operations efficiently, so `decide` handles the full table (Listing 3). With the inserted pixel p at the centre $(3, 3)$:

- `allConfigs 0 256`: for each of the 256 subsets N of the eight neighbours of p, the invariant is 0 on N and on $N \cup \{p\}$. So all 512 subsets of the 3×3 window satisfy the L3 identity, and inserting the centre never changes enc $- C_8 + \chi$.
- `allConfigs ring5 256`: the same for N placed inside the closed 16-pixel ring at Chebyshev distance 2 from p. Every barrier neighbour of p is then already 8-connected to every other through the ring, and every non-barrier run around p lies in a finite region. This is the opposite extreme of global context from the first check.

The bitmask model also reproduces the witness and both negative controls, including the four 4-components of the diamond. In a scratch mutation test (not part of the modules), we replaced 8-adjacency by 4-adjacency in the model's C_8. Both exhaustive checks then failed, so the table depends on the (8,4) pairing. As with the box model, we do not prove that the bitmask model agrees with the semantic definitions. Two contexts are also far from all contexts: the proof of L2 must still handle every intermediate global situation.

Listing 3: The exhaustive configuration check (excerpt from the `Mask` namespace).

```
def inv (B : ℕ) : ℤ := (enc B : ℤ) - c8 B + chi B
def allConfigs (ctx : ℕ) : ℕ → Bool
  | 0 => true
  | n + 1 =>
    let B := ctx ||| spread n
    (inv (B ||| centre) == inv B && inv B == 0) && allConfigs ctx n

set_option maxRecDepth 100000 in
example : allConfigs 0 256 = true := by decide
set_option maxRecDepth 100000 in
example : allConfigs ring5 256 = true := by decide
```

9 A proof of L2, part 1: reduction to a half-plane Jordan lemma

This section reports the 2026-11-08 session on the sealed statement `P1.Insertion`. Section 10 completes the proof. The candidate file `proofs/P1.Insertion.lean` (about 1,950 lines including the 2026-11-09 additions) compiles against the pinned toolchain, and its theorems depend only on `propext`, `Classical.choice` and `Quot.sound`. Write $B' = B \cup \{p\}$ with $p \notin B$, and $W_p(B) = \{q \in B : |q_1 - p_1| \le 1,\ |q_2 - p_2| \le 1\}$ for the part of B in the 3×3 window of p.

Locality of $\Delta\chi$. We give a Lean proof (`euler_insert_local`) that

$$\chi(K(B')) - \chi(K(B)) \;=\; \chi(K(W_p(B) \cup \{p\})) - \chi(K(W_p(B))).$$

The proof does not go through L1. Each of `vertices`, `hEdges` and `vEdges` is a union over pixels, so inserting p adds exactly the cells of p that are not already present. A corner or edge of p can belong to a pixel q only if q is in the window, so whether such a cell is new is decided by $W_p(B)$.

The change in C_8. Let k be the number of 8-components of B that contain an 8-neighbour of p. We give a Lean proof (`C8_insert`) that $C_8(B') = C_8(B) + 1 - k$. The inclusion $B \subseteq B'$ is a graph homomorphism of the 8-adjacency graphs, so it maps components to components. The key lemma is a walk lemma, proved by induction on `Relation.ReflTransGen`. If $v \in B$ reaches y in B', then either $y \neq p$ and v reaches y already in B, or the component of v touches p. From this, the components that touch p all map to the component of p, and the others map injectively onto the rest. The bijection Option{non-touching} $\simeq \pi_0(B')$ gives the count.

The change in the enclosed count. The complement of B' is the complement of B with p removed, and the same walk lemma applies with 4-adjacency. Let c_p be the 4-component of p in $\mathbb{Z}^2 \setminus B$ and let P be the number of enclosed 4-components of $\mathbb{Z}^2 \setminus B'$ that contain a 4-neighbour of p. A component of $\mathbb{Z}^2 \setminus B'$ that contains no 4-neighbour of p has the same support as its image, so it is enclosed exactly when its image is. We give a Lean proof (`enclosed_insert`) that, if B has finitely many enclosed components,

$$\mathrm{enc}(B') = \mathrm{enc}(B) - [\, c_p \text{ enclosed} \,] + P.$$

The finiteness hypothesis is needed because `enclosed` is a `Nat.card`, which is 0 on infinite types.

Finiteness. We give a Lean proof (`finite_enclosed`) that for every finite B the set of enclosed components of $\mathbb{Z}^2 \setminus B$ is finite. Choose M with $|b_1|, |b_2| < M$ for all $b \in B$. A non-barrier pixel v with $v_1 \geq M$, $v_1 \leq -M$ or $|v_2| \geq M$ has a horizontal ray free of B, so by the lemma `infinite_of_ray` of the sealed module its component is infinite. Hence every enclosed component has a representative in the finite box $[-M, M]^2$, and sending a component to its representative is injective. This also discharges the finiteness hypothesis above, and it is clause (1) of the sealed statement `P1.Bridge`. It is stated there in a different form, so we do not claim that statement.

The reduction. Combining the four gives `invariant_insert`:

$$\Delta(\mathrm{enc} - C_8 + \chi) \;=\; \big(P - [\, c_p \text{ enclosed} \,]\big) - (1 - k) + \Delta\chi\big(W_p(B)\big),$$

and `statement_of_jordan` derives `Statement` from one hypothesis:

(J) for every B and $p \notin B$, $P - [\, c_p \text{ enclosed} \,] - (1 - k) + \Delta\chi(W_p(B)) = 0$.

(J) is the real topological content. Let a be the number of maximal 8-connected arcs of barrier pixels on the 8-cycle around p. For $r \geq 1$ non-barrier 4-neighbours we expect $\Delta\chi(W_p) = 1 - a$, and P should equal $a - k + 1$ minus the unbounded piece when c_p is infinite. Each pair of arcs that is already joined by a barrier 8-path away from p closes a digital curve through p and cuts off one more piece. Proving this needs a digital Jordan curve theorem for (8,4) curves [5]. Mathlib does not provide one. The rest of this section narrows (J) down to a single configuration.

A special case of L2. As a check on the reduction, we give a Lean proof (`insertion_hole`) that $\mathrm{enc} - C_8 + \chi$ is unchanged when all four 4-neighbours of p are barrier pixels. In this case no new component contains a 4-neighbour of p, so $P = 0$. The old component $c_p = \{p\}$ is enclosed, by the lemma `isolated` of the sealed module. Every 8-neighbour of p is joined to the north neighbour through the north, east and south neighbours, so $k = 1$. Every corner and edge of p is already present, so $\Delta\chi = 1$. Then (J) reads $0 - 1 - 0 + 1 = 0$. This case covers, for example, filling the centre of the 3×3 ring and of the diamond. It is not the hard case: there, merging and splitting interact through the global shape of B.

Raster-scan order. Say that B is *before* p if every $q \in B$ has $q_2 < p_2$, or $q_2 = p_2$ and $q_1 < p_1$. The invariant of the empty raster is 0, by the control lemmas of the sealed module and $\chi(\emptyset) = 0$. Mathlib's `Finset.induction_on_max_value` with the lexicographic key (q_2, q_1) builds every finite raster by insertions in which B is before p. So if those insertions preserve the invariant, the invariant is 0 on every finite raster, and `Statement` follows (`statement_of_scan`). This is the classical raster-scan idea of connected-component labelling [6].

Insertion in scan order. Write W, SW, S, SE for $p+(-1,0), p+(-1,-1), p+(0,-1), p+(1,-1)$. If B is before p, every pixel above the row of p, and every pixel to the right of p in its row, is free in $B \cup \{p\}$ and has a free ray. We give Lean proofs of the following. The old component c_p is infinite. A new piece that contains E, N or W is infinite: W is 4-adjacent to the free pixel NW. So P is 1 if S is free and its piece is enclosed, and 0 otherwise (`P_scan`). The barrier neighbours of p lie among W, SW, S, SE, and (`euler_scan`)

$$\Delta\chi = [\, W, S, SW \notin B \,] + [\, S, SE \notin B \,] - [\, S \notin B \,].$$

Table 3 lists the five cases. In four of them k and P are fixed by local arguments, and the invariant is preserved. In the fifth, S is free, $SE \in B$ and $L \in \{W, SW\}$ meets B. Then $\Delta\chi = -1$, $k \in \{1, 2\}$, and we give a Lean proof (`k_core`) that $k = 1$ exactly when L and SE are 8-connected in B.

Case (scan order)	$\Delta\chi$	k	P	reason for P
$S \in B$	0	1	0	S is not free
$S, SE \notin B$, $W, SW \notin B$	1	0	0	$S \to SE \to E$, free ray
$S, SE \notin B$, $L \cap B \neq \emptyset$	0	1	0	$S \to SE \to E$
$S \notin B$, $SE \in B$, $W, SW \notin B$	0	1	0	$S \to SW \to W \to NW$
$S \notin B$, $SE \in B$, $L \cap B \neq \emptyset$	-1	1 or 2	$2 - k$	half-plane Jordan lemma

Table 3: Insertion of p after all of B in raster-scan order. In every row the invariant changes by $P - (1 - k) + \Delta\chi$ (the old component of p is never enclosed), so it is preserved exactly when $P + k + \Delta\chi = 1$. Rows 1–4 were proved by local arguments on 2026-11-08. Row 5 needs $P + k = 2$, which is the half-plane Jordan lemma (crossing parity, Section 9, and the crack boundary, Section 10).

One direction by crossing parity. We give a Lean proof (`senc_of_join`) of the direction "L and SE 8-connected $\Rightarrow$ the component of S is finite". Take an 8-walk γ in B from L to SE and close it into the cycle $\Gamma = (p, \gamma, p)$. For a pixel v let $N(v)$ count the steps $a \to b$ of Γ that join column v_1 to column $v_1 + 1$ and whose endpoint in column v_1 lies strictly above v. Each of the following facts reduces to an identity, modulo 2, for a single step of Γ. Each such identity is a statement in linear integer arithmetic, closed by `omega`. A generic lemma (`dsum_parity`) then adds them up along the walk:

- if v and $v + (0,1)$ are free, $N(v) \equiv N(v + (0,1))$: a step could distinguish them only through the free pixel $v + (0,1)$;
- if v and $v + (1,0)$ are free, $N(v) + N(v + (1,0))$ is the number of times Γ enters or leaves the half-column $\{(v_1 + 1, t) : t > v_2\}$, which is even because Γ is closed;
- $N(v)$ is even if v is above the row of p, to the left or right of all of Γ, or below all of Γ (then $N(v)$ counts every crossing of a full vertical line, which is even);

- $N(S)$ is odd: the only step of Γ that reaches column $x+1$ from a pixel of column x at height at least y is $SE \to p$, because B is before p.

So N is odd on the whole free 4-component of S, which therefore lies in a finite box.

Half of the bridge identity. In the fifth case the invariant changes by $P + k - 2$. If L and SE are connected then $k = 1$ and, by `senc_of_join`, $P = 1$. Otherwise $k = 2$ and $P \geq 0$. So every scan-order insertion changes the invariant by 0 or +1 (`scan_step_mono`). Since the empty raster has invariant 0, we give a Lean proof (`invariant_nonneg`, `enclosed_ge`) that for every finite raster

$$\mathrm{enc}(B) \;\geq\; C_8(B) - \chi(K(B)).$$

The invariant of B counts the "bad" scan steps, where S is enclosed although L and SE lie in different 8-components. `statement_of_le` derives `Statement` from the reverse inequality.

What remained on 2026-11-08. `statement_of_halfplane_jordan` derives `Statement` from the following hypothesis. From the same hypothesis, `bridge_of_halfplane_jordan` derives the equation $\mathrm{enc}(B) = C_8(B) - \chi(K(B))$ of L3 for every finite B.

> **Half-plane Jordan lemma.** Let B be before p, with $S \notin B$, $SE \in B$ and $W \in B$ or $SW \in B$. Then the 4-component of S in $\mathbb{Z}^2 \setminus (B \cup \{p\})$ is finite if and only if W or SW lies in the 8-component of SE in B.

One direction says that an 8-path in B from L to SE, closed up through p, cuts S off from infinity. It has a Lean proof (above). The other says that if no such path exists, S escapes around the 8-component of SE (`statement_of_enc_join` derives `Statement` from this direction alone). Both are instances of the digital Jordan curve theorem for (8,4) connectivity [5, 4]. The second direction needs a different tool, boundary following. Section 10 gives it. The four-pixel diamond shows that the case cannot be avoided by a cleverer choice of insertion order. Each of its pixels, inserted last in any of the eight scan orders, falls into row 5 with L and SE joined.

The key statements, as they appear in the candidate file:

```
theorem euler_insert_local (B : Finset Pixel) (p : Pixel) (hp : p ∉ B) :
    euler (insert p B) - euler B = euler (insert p (win B p)) - euler (win B p)

theorem C8_insert (hp : p ∉ B) :
    (C8 (insert p B) : ℤ) = C8 B + 1 - Nat.card {c // TouchC B p c}

theorem enclosed_insert (hp : p ∉ B)
    (hfin : Finite {c : (complementGraph B).ConnectedComponent // IsEnclosed c}) :
    (enclosed (insert p B) : ℤ) = enclosed B - (if IsEnclosed (cp B p hp) then 1 else 0)
      + Nat.card {x : {c : (complementGraph (insert p B)).ConnectedComponent // IsEnclosed c} //
          TouchPC B p x.1}

theorem statement_of_halfplane_jordan
    (hJ : ∀ (B : Finset Pixel) (p : Pixel), p ∉ B → Before B p →
      (p.1, p.2 - 1) ∉ B → (p.1 + 1, p.2 - 1) ∈ B →
      ((p.1 - 1, p.2) ∈ B ∨ (p.1 - 1, p.2 - 1) ∈ B) →
      (SEnc B p ↔ JoinLR B p)) :
    Statement

theorem senc_of_join (hb : Before B p) (hS : (p.1, p.2 - 1) ∉ B) (hJ : JoinLR B p) :
    SEnc B p
```

```
theorem invariant_nonneg (B : Finset Pixel) : 0 ≤ invariant B

theorem statement_of_le
    (h : ∀ B : Finset Pixel, (enclosed B : ℤ) ≤ C8 B - euler B) : Statement

lemma cross_horiz (v a b : Pixel) (hadj : G8.Adj a b)
    (ha1 : ¬ (a.1 = v.1 ∧ a.2 = v.2)) (ha2 : ¬ (a.1 = v.1 + 1 ∧ a.2 = v.2))
    (hb1 : ¬ (b.1 = v.1 ∧ b.2 = v.2)) (hb2 : ¬ (b.1 = v.1 + 1 ∧ b.2 = v.2)) :
    (cross v a b + cross (v.1 + 1, v.2) a b
      - (if a.1 = v.1 + 1 ∧ v.2 < a.2 then 1 else 0) - (if b.1 = v.1 + 1 ∧ v.2 < b.2 then 1 else
    0))
      % 2 = 0 := by
  have := g8_coords hadj
  unfold cross
  simp only
  split_ifs <;> omega

lemma walk_lemma (v : (B : Set Pixel)) (y : ((insert p B : Finset Pixel) : Set Pixel))
    (h : (barrierGraph (insert p B)).Reachable (incl B p v) y) :
    (∃ y0 : (B : Set Pixel), y.1 = y0.1 ∧ (barrierGraph B).Reachable v y0) ∨ Touch B p v := by
  rw [SimpleGraph.reachable_iff_reflTransGen] at h
  induction h with
  | refl => exact Or.inl ⟨v, rfl, SimpleGraph.Reachable.refl _⟩
  | @tail y z _ hyz ih =>
    rcases ih with ⟨y0, hy, hr⟩ | ht
    · rw [barrier_adj_iff] at hyz
      by_cases hz : z.1 = p
      · right
        exact ⟨y0, hr, by rw [← hz, ← hy]; exact hyz.symm⟩
      · have hzB : z.1 ∈ (B : Set Pixel) := by
          have := z.2
          simp only [Finset.coe_insert, Set.mem_insert_iff] at this
          exact this.resolve_left hz
        left
        refine ⟨⟨z.1, hzB⟩, rfl, hr.trans (SimpleGraph.Adj.reachable ?_)⟩
        rw [barrier_adj_iff]; rw [← hy]; exact hyz
    · exact Or.inr ht
```

10 A proof of L2, part 2: escape along the crack boundary

This section reports the 2026-11-09 session. It proves the remaining direction of the half-plane Jordan lemma. With it, the file `proofs/P1.Insertion.lean` contains `Candidate.main : ViridisStatements.P1.I` The project's checker `viridis-lean-check` ran all five of its stages (denylist, compilation, statement match, axioms, kernel replay) on this file and passed it, with axioms `propext`, `Classical.choice` and `Quot.sound`. That is a local preview; the verifier's own report is the certification page.

The claim. Let B be before p, with $S \notin B$, $SE \in B$, and suppose that neither W nor SW lies in the 8-component C of SE in B. We show that S is joined to $E = p + (1, 0)$ by a 4-path in $\mathbb{Z}^2 \setminus (B \cup \{p\})$. Since E has a free ray, the component of S is infinite (`join_of_senc`, by contraposition). Two facts about C make the problem local:

- no pixel of $B \setminus C$ lies within Chebyshev distance 1 of C, since it would be 8-adjacent to C and hence in C. So every pixel near C but not in C is free, apart from p itself;

- the only pixel of C within Chebyshev distance 1 of p is SE. The other candidates are W and SW (not in C by assumption), S (free), and pixels at or after p in scan order (not in B). So p touches C only at the corner shared by SE.

Cracks and their successor. A *crack* of a finite set C is a pair (c, d) with $c \in C$, $d \in \{(\pm 1, 0), (0, \pm 1)\}$ and $c + d \notin C$: a unit edge of $K(C)$ with C on one side and the outside pixel $c + d$ on the other. Let $t = \mathrm{rot}(d) = (-d_2, d_1)$. The successor of (c, d) follows the (8,4) rule:

$$\mathrm{next}(c, d) = \begin{cases} (c + d + t,\ -t) & \text{if } c + d + t \in C \quad \text{(turn outward, diagonal step in } C\text{)}, \\ (c + t,\ d) & \text{else if } c + t \in C \quad \text{(straight on)}, \\ (c,\ t) & \text{otherwise} \quad \text{(turn around } c\text{)}. \end{cases}$$

We give Lean proofs that next maps cracks to cracks (`next_mem`) and that it has an explicit left inverse prev on cracks, given by the mirror rule (`prev_next`). So next is a permutation of the finite set of cracks. By a pigeonhole argument (`orbit`) every crack x_0 has a least period $n > 0$ with $\mathrm{next}^n(x_0) = x_0$.

The orbit from S. Take $x_0 = (SE, (-1, 0))$, whose outside pixel is S, and $x_E = (SE, (0, 1))$, whose outside pixel is E. A direct computation gives $\mathrm{next}(x_E) = x_0$: the diagonal pixel is $p \notin C$ and the side pixel is $S \notin C$, so the boundary turns around SE. Since $\mathrm{next}(\mathrm{next}^{n-1} x_0) = x_0 = \mathrm{next}(x_E)$, injectivity gives $\mathrm{next}^{n-1} x_0 = x_E$. Minimality of n gives $\mathrm{next}^k x_0 \neq x_E$ for $k < n - 1$. Neither fact uses any planar topology.

One step is one short free path. Let $x = (c, d)$ be a crack other than x_E. We give a Lean proof (`crack_step`) that the outside pixels of x and of $\mathrm{next}(x)$ are joined by a free 4-path of length at most 2. In the first case of the rule the outside pixel is unchanged. In the second it moves to the 4-adjacent pixel $c + d + t$. In the third it moves from $c + d$ to $c + t$ through the diagonal pixel $c + d + t$. Every pixel involved is within distance 1 of C and not in C, so it is free unless it is p. The outside pixels are 4-adjacent to C, and p is not. The diagonal pixel of the third case equals p only if $c = SE$ and $d + t = (-1, 1)$, that is, only for $x = x_E$. Each of these local facts is linear integer arithmetic over the four directions and is closed by `omega`.

Chaining the steps for $k = 0, \ldots, n - 2$ gives a free 4-path from S to E (`crack_escape`). This proves the remaining direction. Then `statement_of_enc_join` gives `Statement`, and `bridge_of_halfplane_jordan` gives the bridge equation for every finite raster (`bridge_eq`).

Why this was the right tool. The earlier sessions considered two routes: boundary following, and the lemma that the border of a finite 4-connected set is 8-connected. Formalizing boundary following usually means proving that the traced sequence closes up and visits the right pixels. The permutation view avoids both. Closing up is pigeonhole plus injectivity, and the only global information needed is that the step into x_0 comes from x_E. That step is the one that passes the corner at p. We also tried, on paper, an induction that fills free pixels of the component of S one at a time. It fails on annular components: filling any pixel of an annulus around a one-pixel island merges the island with the outer barrier.

The new definitions and the final theorems:

```
def next (C : Finset Pixel) (x : Pixel × Pixel) : Pixel × Pixel :=
  if (x.1.1 + x.2.1 - x.2.2, x.1.2 + x.2.2 + x.2.1) ∈ C then
    ((x.1.1 + x.2.1 - x.2.2, x.1.2 + x.2.2 + x.2.1), (x.2.2, -x.2.1))
```

```
  else if (x.1.1 - x.2.2, x.1.2 + x.2.1) ∈ C then ((x.1.1 - x.2.2, x.1.2 + x.2.1), x.2)
  else (x.1, (-x.2.2, x.2.1))

lemma prev_next (C : Finset Pixel) (x : Pixel × Pixel) (hx : x ∈ cracks C) :
    prev C (next C x) = x

lemma orbit {α : Type*} (f g : α → α) (s : Finset α) (hs : ∀ x ∈ s, f x ∈ s)
    (hg : ∀ x ∈ s, g (f x) = x) (x0 : α) (h0 : x0 ∈ s) :
    ∃ n, 0 < n ∧ f^[n] x0 = x0 ∧ ∀ k, 0 < k → k < n → f^[k] x0 ≠ x0

theorem crack_escape (hc0 : c0 ∈ C)
    (hfree : ∀ c ∈ C, ∀ q : Pixel, q ∉ C → Near q c → q ≠ p → q ∉ B)
    (hp : p = (c0.1 - 1, c0.2 + 1))
    (hpC : ∀ c ∈ C, Near p c → c = c0)
    (hS : (c0.1 - 1, c0.2) ∉ C) (hE : (c0.1, c0.2 + 1) ∉ C) :
    FR B p (c0.1 - 1, c0.2) (c0.1, c0.2 + 1)

theorem join_of_senc (hp : p ∉ B) (hb : Before B p) (hS : (p.1, p.2 - 1) ∉ B)
    (hSE : (p.1 + 1, p.2 - 1) ∈ B) (hs : SEnc B p) : JoinLR B p

theorem bridge_eq (B : Finset Pixel) : (enclosed B : ℤ) = C8 B - euler B

theorem main : ViridisStatements.P1.Insertion.Statement :=
  statement_of_enc_join fun B p hp hb hS hSE _ hs => join_of_senc B p hp hb hS hSE hs
```

Here `FR B p a b` says that a and b are free in $B \cup \{p\}$ and joined by a 4-path there, and `Near q c` says that q and c are at Chebyshev distance at most 1.

11 Computational evidence for L2 and L3

The script `code/bridge_check.py` is independent of both Lean models. It was rewritten for this revision, because the first draft's script is not part of the inputs available to this session. It pads each raster by one pixel, labels the 4-components of the complement and the 8-components of B by depth-first search, and computes $V - E + F$ directly. A complement component counts as enclosed when it does not touch the padded border. It found no failures (`results/bridge_check.json`):

- L3 on all $2^{16} = 65{,}536$ subsets of a 4×4 box;
- L3 on 3,000 random rasters of side up to 12 with random density (seed 20261104);
- L2 on 10,496 insertions: each of the 256 configurations of the eight neighbours of p, alone and in 40 random contexts at Chebyshev distance between 2 and 6 from p.

This is evidence that the proposed statements are true and that their conventions are the intended ones. It is not a proof.

For the 2026-11-08 reduction, the script `code/core_check.py` (standard library only, about 3 seconds) tests the exact hypothesis of `statement_of_halfplane_jordan`. It draws 4,000 random rasters (side 3 to 9, seed 20261108). At every candidate pixel p it restricts the raster to the pixels before p and keeps the configurations of row 5 of Table 3. In all 17,344 such configurations (7,924 enclosed, 9,420 not enclosed), the finiteness of the component of S agrees with the 8-connection of L and SE. The script also checks the full invariant on 15,281 scan-order insertions, with no failures (`results/core_check.json`). As a mutation test, a variant that ignores SW in the connection test reports 3,662 disagreements. So the check can tell the two sides apart.

12 Limitations

- The theorem concerns the combinatorial counts $V - E + F$ as defined in the sealed module. The development does not show that this number is the Euler characteristic of the topological space $K(B) \subset \mathbb{R}^2$. That identification is standard but is not formalised here, and L2/L3 are designed not to need it.
- Only the 8-connected weighting of the bit quads is treated. The 4-connected variant, with $+2Q_D$, is not formalised.
- We give a Lean proof of L2 and of the bridge equation enc $= C_8 - \chi$ (`bridge_eq`). The sealed L3 statement `P1.Bridge` is a separate task with its own exact form, and we do not claim it here. The `FaceCount` hypothesis of *Trees Cannot Cut* is about a corridor graph, not a raster. Discharging it also needs a raster-to-graph correspondence, which is not formalised. The heuristic description of (J) in terms of arcs on the 8-cycle is a conjecture and has no Lean proof. The proof of L2 goes through scan order instead.
- The bounded-box model used for the `decide` checks is not proved to agree with the semantic definitions. Only the four semantic control lemmas link the two.
- The `decide` table of all 256 neighbourhood configurations uses only two global contexts: none, and a closed surrounding ring. The Python check adds 40 random contexts per configuration. Neither replaces a proof of L2.
- Attempts to use the Aristotle prover (finiteness clause of L3 on 2026-11-04; the full L2 statement on 2026-11-08; a status check on 2026-11-09) failed with an invalid API key, as in earlier sessions. No automated proof search was done, and every proof in the file was written by hand.
- L3 relates enclosed components to $C_8 - \chi$, not to the circuit rank of a corridor graph. A paper that uses L3 in place of `FaceCount` must state which raster-to-graph correspondence it uses. The 2×2 block shows that the two notions differ.

13 Reproducibility

The L1 proof file `proofs/P1.BitQuad.lean` and the script `code/bitquad_check.py` belong to the package of the 2026-10-28 run. It imports only `Mathlib` and the sealed module `ViridisStatements.P1.BitQuad` (sha256 `cd3cdeba48f6...8e23`), and it builds with toolchain `leanprover/lean4:v4.28.0` and Mathlib commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365`. The check takes about 20 seconds. The brute-force script runs with the Python 3 standard library: `python3 code/bitquad_check.py results/bitquad_4x4.json`. No external data are used.

The proposals `proposals/P1.Insertion.lean` and `proposals/P1.Bridge.lean` each compile with `lean` against the same toolchain and Mathlib commit, in about 100 and 60 seconds respectively, including all `decide` checks (among them the two 256-configuration tables) and the semantic control proofs. Both import only `Mathlib`. The evidence for L2/L3 is reproduced by `python3 -I code/bridge_check.py results/bridge_check.json` (standard library only, about 6 seconds). The proposals, the evidence script and its results belong to the package of the 2026-11-04 run.

The file `proofs/P1.Insertion.lean` (2026-11-09 package) imports only `Mathlib` and the sealed module `ViridisStatements.P1.Insertion` (sha256 `800bd9b9fec5...c734`). It builds with

toolchain `leanprover/lean4:v4.28.0` and the Mathlib commit above, and compiles in about 30 seconds with `lean`. Run `viridis-lean-check proofs/P1.Insertion.lean P1.Insertion` to check it (about 50 seconds). On our run it passed all five stages and reported the axioms `propext`, `Classical.choice` and `Quot.sound`. The script `code/core_check.py` belongs to the 2026-11-08 session and is not part of this package. The proof does not depend on it.

Changelog

- 2026-10-28 (P1-T001): first version of the paper. Lean proof of L1 (`P1.BitQuad`), proof architecture, brute-force check on the 4×4 box, and discussion of the role of L1 in L2/L3.
- 2026-11-03 (P1-T002): new title and abstract. Added the proposed statements `P1.Insertion` (L2) and `P1.Bridge` (L3) with their conventions, controls (box-model `decide` checks and Lean proofs of the control values for the semantic definitions), brute-force evidence for L2/L3, the reference to Rosenfeld (1970), and updated limitations and reproducibility.
- 2026-11-04 (P1-T002, second attempt): the verifier had rejected the first draft of the proposals because the import of the L1 module did not resolve in its environment. The proposal modules now restate the L1 definitions verbatim and import only Mathlib. Added the bitmask model with `decide` checks of all 256 neighbourhood configurations in two contexts, and rewrote the evidence script (new seed). Updated abstract, limitations and reproducibility.
- 2026-11-08 (P1-T003): new section on a reduction of L2 (Section 9): Lean proofs of locality of $\Delta\chi$, of $\Delta C_8 = 1 - k$, of finiteness of the enclosed components, of the change in the enclosed count, of L2 from the Jordan-type identity (J), and of L2 in the special case where the four 4-neighbours of p are barrier. Then the raster-scan reduction: L2 from scan-order insertions, the explicit case analysis of Table 3, and L2 from a half-plane digital Jordan lemma, which is checked by brute force (`code/core_check.py`). A Lean proof of one direction of that lemma by crossing parity, and of the inequality enc $\geq C_8 - \chi$ for every finite raster. Updated the title, abstract, introduction, limitations and reproducibility. L2 remains unproved.
- 2026-11-09 (P1-T003): new Section 10. Lean proof of the remaining direction of the half-plane Jordan lemma by following the crack boundary (a permutation of the finite set of cracks), and hence a Lean proof of the sealed statement `P1.Insertion` (`Candidate.main`) and of the bridge equation enc $= C_8 - \chi$ (`bridge_eq`). Updated the title, abstract, introduction, Table 3, limitations and reproducibility.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Stephen B. Gray. Local properties of binary images in two dimensions. *IEEE Transactions on Computers*, C-20(5):551–561, 1971. doi: 10.1109/T-C.1971.223289.

[3] Justin D. Hart. Trees cannot cut: Circuit rank as the design variable reconciling habitat connectivity and fuel discontinuity in managed forest landscapes. Zenodo, Viridis LLC, 2026.

[4] T. Yung Kong and Azriel Rosenfeld. Digital topology: Introduction and survey. *Computer Vision, Graphics, and Image Processing*, 48(3):357–393, 1989. doi: 10.1016/0734-189X(89)90147-3.

[5] Azriel Rosenfeld. Connectivity in digital pictures. *Journal of the ACM*, 17(1):146–160, 1970. doi: 10.1145/321556.321570.

[6] Azriel Rosenfeld and John L. Pfaltz. Sequential operations in digital picture processing. *Journal of the ACM*, 13(4):471–494, 1966. doi: 10.1145/321356.321357.

[7] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.